

UGC Video Generation Platform

Minimal Agent Architecture Design Document

Author

Souvik Ojha

Version

1.0

Executive Summary

This project is a conversational AI application capable of generating short-form UGC-style marketing videos from natural language prompts and product URLs.

Users interact through a simple chat interface. When a user provides a product URL, the system analyzes the product, generates a trend-aware creative concept, assembles media assets, renders a short video, and returns the final video URL directly in the chat.

The system follows an **AI-Organized, Not AI-Generated** philosophy.

Instead of generating videos with generative AI models, the platform uses AI to make creative decisions while assembling existing media assets through a deterministic rendering pipeline.

This approach delivers:

- Faster generation times
 - Lower infrastructure costs
 - More predictable outputs
 - Better trend alignment
 - Higher reliability
-

Problem Statement

Example User Message:

"I'm building CalAI, a calorie tracking app.

<https://calai.app>"

Expected Workflow:

1. Understand the product
2. Analyze audience and value proposition
3. Create a social-media-style creative concept
4. Select relevant assets
5. Assemble a short-form UGC video
6. Return video URL

Example Response:

Assistant:

"Analyzing your product..."

Assistant:

"Creating your video..."

Assistant:

"Your video is ready:

<https://cdn.example.com/videos/calai.mp4>"

Design Philosophy

The objective is not to build a complex multi-agent system.

The objective is to build a system that:

- Works end-to-end
- Produces funny outputs
- Feels modern
- Generates videos quickly
- Can be implemented within assignment timelines

The architecture intentionally minimizes agent count.

Everything that can be deterministic remains deterministic.

AI is only responsible for:

- Product understanding
- Creative ideation
- Creative evaluation

Everything else is handled through traditional software systems.

Technology Stack

Frontend

Framework: Next.js 15

Language: TypeScript

UI: TailwindCSS

Components: ShadCN UI

Data Fetching: React Query

Real-Time Updates: Server Sent Events (SSE)

Backend

Framework: FastAPI

Language: Python 3.12

Agent Framework

Agno

LLM

Gemini 2.5 Flash

Alternative:

GPT-4o Mini

Video Rendering

FFmpeg

Database

PostgreSQL

Storage

Supabase Storage

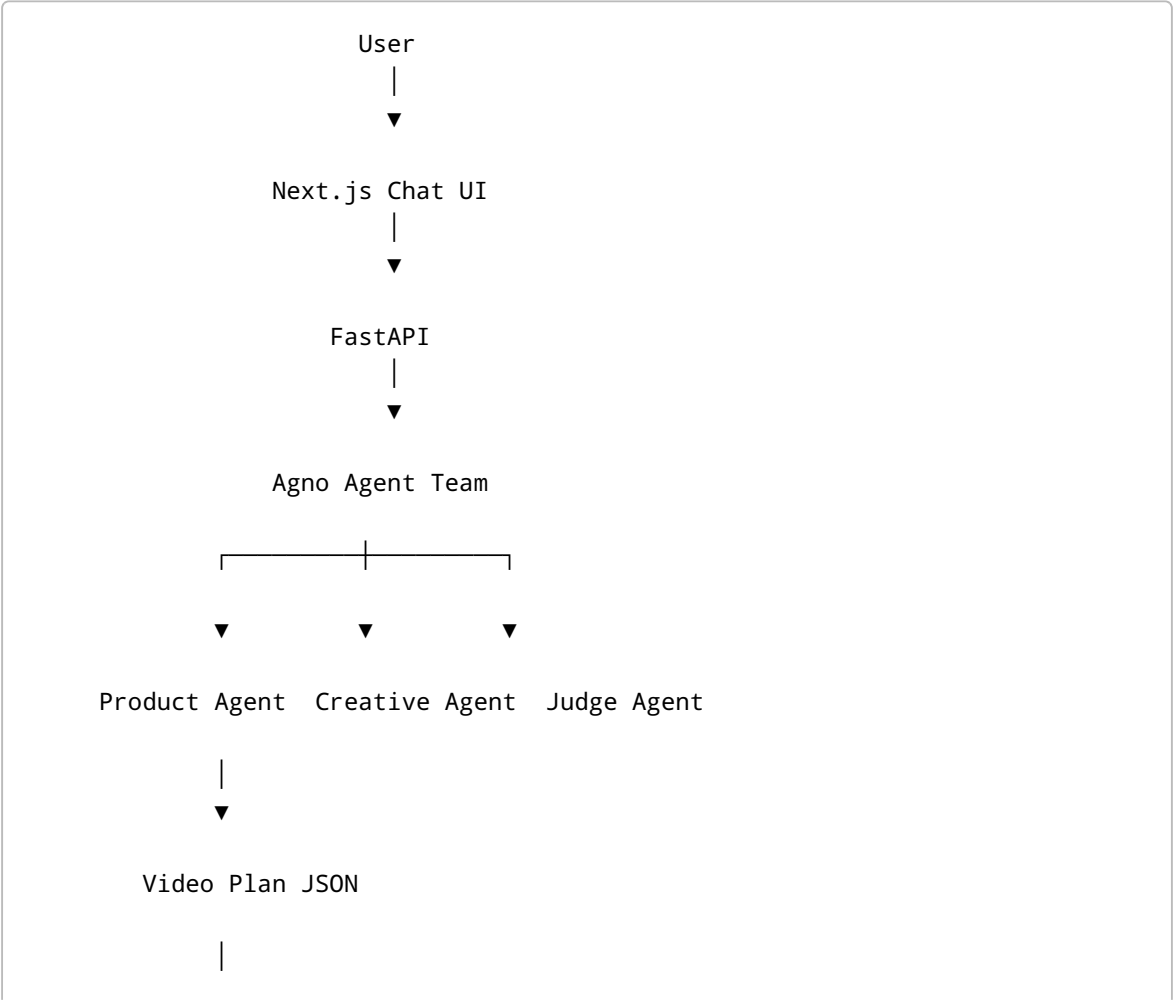
Asset Providers

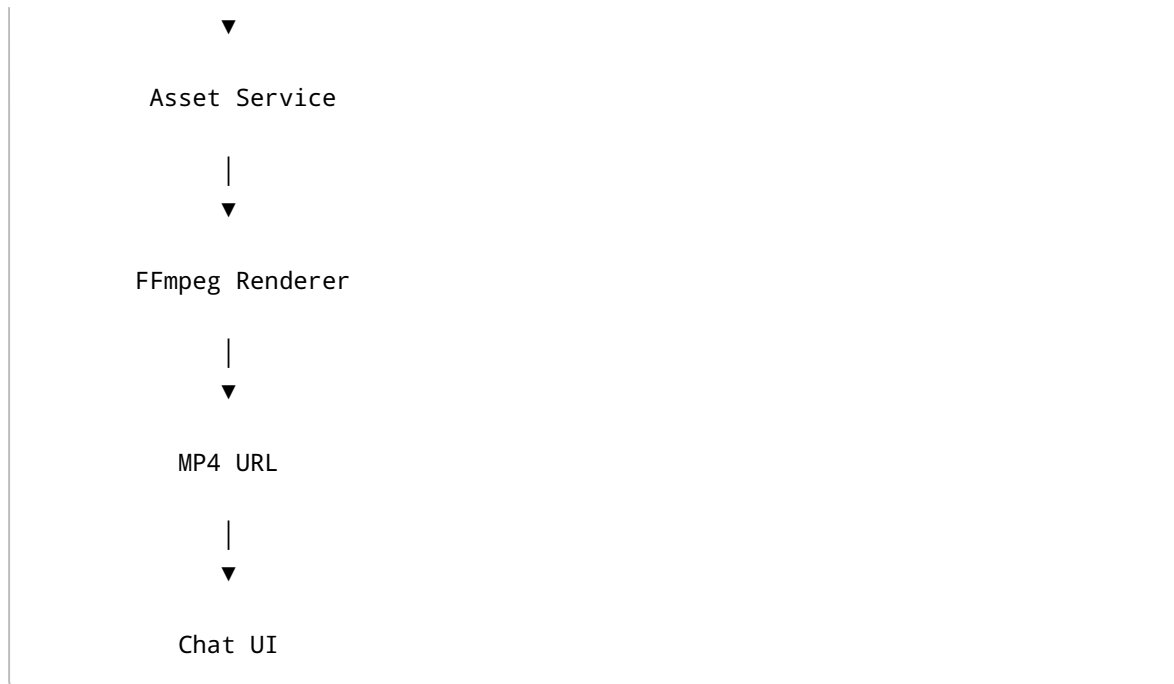
Video Assets: Pexels API

GIF Assets: Tenor API

Audio Assets: Curated Local Audio Library

System Architecture





Why Only Three Agents?

Most proposed solutions introduce many unnecessary agents:

- Trend Agent
- Asset Agent
- Audio Agent
- Storyboard Agent
- Rendering Agent

These increase complexity while providing little value.

This project only requires three reasoning steps:

1. Understand the product
2. Generate creative concepts
3. Select the best concept

Everything else is execution.

Therefore:

Product Agent

+ Creative Agent

+ Judge Agent

is sufficient.

Agent Design

Product Agent

Purpose

Understand the product and create a structured product brief.

Inputs

- User message
- Website content

Responsibilities

- Extract URL
- Read website
- Identify product category
- Identify target audience
- Identify user pain points
- Identify product value proposition

Example Output

```
{ "product": "CalAI", "category": "Fitness", "targetAudience": "Gym Enthusiasts", "painPoint": "Manual calorie tracking", "valueProposition": "AI-powered calorie estimation" }
```

Tools

Website Scraper Service

Creative Agent

Purpose

Generate multiple marketing concepts.

Input

Product Brief

Responsibilities

Generate:

- Hook
- Meme format

- Text overlays
- GIF concept
- Audio style
- Story concept

Example Output

```
[ { "hook": "Bro said he only had a snack",
```

```
  "gif":
    "rock eyebrow",

  "ending":
    "CalAI found 2487 calories"
```

```
},
```

```
{ "hook": "POV: You're on a cut",
```

```
  "gif":
    "crying cat",

  "ending":
    "The app saw your midnight biryani"
```

```
},
```

```
{ "hook": "Gym bro math",
```

```
  "gif":
    "confused math lady",

  "ending":
    "Calories don't work like that"
```

```
}]
```

Output Count

3 concepts

This allows comparison and selection.

Judge Agent

Purpose

Select the strongest concept.

Input

Creative Concepts

Responsibilities

Evaluate:

- Humor
- Virality
- Trend relevance
- Product relevance
- Audience fit

Example Output

```
{ "winner": 2, "score": 91, "reason": "Most relatable fitness meme with strong social media appeal." }
```

Final Output

Selected Creative Concept

Agno Team Configuration

The entire AI system is represented by a single Agno Team.

Example Flow:

User Input ↓ Product Agent ↓ Creative Agent ↓ Judge Agent ↓ Video Plan

Simple.

Predictable.

Easy to debug.

Video Plan Contract

After AI processing, the system generates a deterministic Video Plan.

This becomes the contract between AI and the rendering pipeline.

Example:

```
{ "duration": 8,  
  
  "backgroundSearch": "gym workout",  
  
  "gifSearch": "rock eyebrow",  
  
  "audioCategory": "upbeat",  
  
  "scenes": [ { "start": 0, "end": 4, "text": "Bro said he only had a snack" }, { "start": 4, "end": 8, "text": "CalAI  
found 2487 calories" } ] }
```

The rendering system never calls the LLM.

It simply executes this plan.

Supporting Services

Website Scraper Service

Purpose:

Extract website information.

Implementation:

Playwright

Responsibilities:

- Visit URL
- Extract title
- Extract description
- Extract hero content
- Extract product information

Output:

Clean website summary

Asset Service

Purpose:

Acquire media assets.

Responsibilities:

Video Search

Pexels API

GIF Search

Tenor API

Audio Selection

Local Audio Library

Input:

Video Plan

Output:

Asset URLs

Audio Library

The system maintains a small curated library.

Examples:

upbeat_01.mp3 upbeat_02.mp3 upbeat_03.mp3

meme_01.mp3 meme_02.mp3

dramatic_01.mp3

This avoids dependence on external music APIs.

Video Rendering Pipeline

Rendering is completely deterministic.

No AI involvement.

Video Structure

Resolution:

1080 x 1920

Aspect Ratio:

9:16

Duration:

5–10 seconds

Layer Structure

Layer 1

Background Video

Layer 2

Text Overlay

Layer 3

GIF Overlay

Layer 4

Audio Track

Composition

Background Video

+ Text Overlay

+ GIF Overlay

+ Audio = Final MP4

FFmpeg Pipeline

Input Assets

Background Video

GIF Overlay

Audio Track

Text Elements

Rendering

FFmpeg Filter Graph

Output

Final MP4

Example Output:

calai_final.mp4

User Experience

General Chat

User:

Hi

Assistant:

Hey! I can create short-form UGC marketing videos from product URLs.

User:

What can you do?

Assistant:

I can analyze products and generate social-media-style UGC videos using trending formats, GIFs, and audio.

Video Generation

User:

<https://calai.app>

Assistant:

Analyzing product...

Assistant:

Generating concepts...

Assistant:

Rendering video...

Assistant:

Your video is ready:

<https://cdn.example.com/videos/calai.mp4>

Database Design

chats

id

created_at

messages

id

chat_id

role

content

created_at

video_jobs

id

chat_id

status

progress

product_url

created_at

generated_videos

id

job_id

video_url

duration

created_at

Evaluation Criteria Alignment

End-to-End Functionality

URL → Analysis → Video → URL

Fully automated workflow.

Humor and Creativity

Creative Agent generates multiple concepts.

Judge Agent selects strongest concept.

Trend Relevance

Concepts are generated using social-media-first prompts.

Trending GIFs and meme structures are prioritized.

Speed

No AI media generation.

Only AI decision making.

FFmpeg handles rendering.

Target generation time:

15–30 seconds.

Technical Quality

Modern stack:

- Next.js
- FastAPI
- Agno
- FFmpeg
- PostgreSQL
- Supabase

Clean separation of reasoning and execution.

Conclusion

This architecture intentionally prioritizes simplicity, reliability, and quality of output.

The system uses only three agents:

1. Product Agent
2. Creative Agent
3. Judge Agent

All media acquisition and rendering remain deterministic services.

This design minimizes complexity while maximizing the probability of producing funny, trend-aware, high-quality UGC videos within a fast and reliable user experience.